

Dossier de projet — Trouve ton artisan

Bilal Bolat

Développeur Web

Centre Européen de Formation

Date : 21 juin 2026

Sommaire

1. Contexte du projet
 2. Expression des besoins
 3. Contraintes
 4. Livrables attendus
 5. Maquettes Figma
 6. Base de données — MCD et MLD
 7. Sécurité — mesures mises en place
 8. Veille sur les vulnérabilités de sécurité
 9. Liens du projet
-

1. Contexte du projet

Présentation de l'entreprise

La région Auvergne-Rhône-Alpes est une collectivité territoriale française couvrant 12 départements dans le quart sud-est de la France. Elle possède des bureaux à Lyon et à Clermont-Ferrand. Ce projet a été mené en collaboration avec l'antenne de Lyon, située au 101 cours Charlemagne, CS 20033, 69269 Lyon Cedex 02.

L'artisanat occupe une place importante dans cette région : près d'un tiers des entreprises y sont des entreprises artisanales, soit environ 221 000 entreprises recensées en 2021. C'est l'une des régions les plus artisanales de France. La région souhaite valoriser et pérenniser cet engagement en offrant une vitrine numérique aux artisans locaux.

Objectif du projet

La région Auvergne-Rhône-Alpes a mandaté la création d'une plateforme web dédiée aux artisans de la région, intitulée "**Trouve ton artisan**". L'objectif principal est de permettre aux particuliers de trouver facilement un artisan qualifié et de le contacter directement via un formulaire de contact pour obtenir des renseignements, des devis ou des informations sur les prestations proposées.

2. Expression des besoins

Besoins fonctionnels

La plateforme doit permettre aux utilisateurs de :

- Naviguer entre les différentes catégories d'artisans : Bâtiment, Services, Fabrication et Alimentation
- Rechercher un artisan par son nom grâce à une barre de recherche
- Consulter la liste des artisans filtrée par catégorie ou par recherche
- Accéder à la fiche complète d'un artisan (nom, photo, spécialité, localisation, note, à propos, site web)
- Contacter un artisan directement via un formulaire de contact (nom, email, objet, message)
- Découvrir les artisans mis en avant chaque mois sur la page d'accueil
- Comprendre le fonctionnement du site grâce à une section explicative en 4 étapes

Besoins non fonctionnels

- **Accessibilité** : le site doit être conforme à la norme WCAG 2.1 et accessible à tous les profils d'utilisateurs (jeunes, personnes âgées, personnes en situation de handicap)
 - **Responsive** : le site doit être conçu en mobile first et fonctionner de manière optimale sur tous les supports (mobile, tablette, ordinateur)
 - **Performance** : les pages doivent se charger rapidement
 - **Sécurité** : les bonnes pratiques de sécurité web doivent être appliquées
 - **Cohérence visuelle** : le site doit s'intégrer à l'environnement numérique de la région Auvergne-Rhône-Alpes
-

3. Contraintes

Contraintes techniques

- **Frontend** : ReactJS, Bootstrap 5, Sass
- **Backend (API)** : Node.js, Express, Sequelize
- **Base de données** : MySQL ou MariaDB
- **Versionning** : Git et GitHub
- **Éditeur** : Visual Studio Code
- **Maquettage** : Figma

- **Police** : Graphik (police officielle de la région)
- **Palette de couleurs** : #f1f8fc, #0074c7, #00497c, #384050, #cd2c2e, #82b864

Contraintes organisationnelles

- Le site doit être hébergé et accessible en ligne
- Le code doit être propre, commenté et correctement indenté
- Le code HTML doit être conforme aux standards W3C
- Les pages légales (mentions légales, données personnelles, accessibilité, cookies) seront remplies ultérieurement par un cabinet spécialisé
- Les catégories du menu doivent être alimentées depuis la base de données

4. Livrables attendus

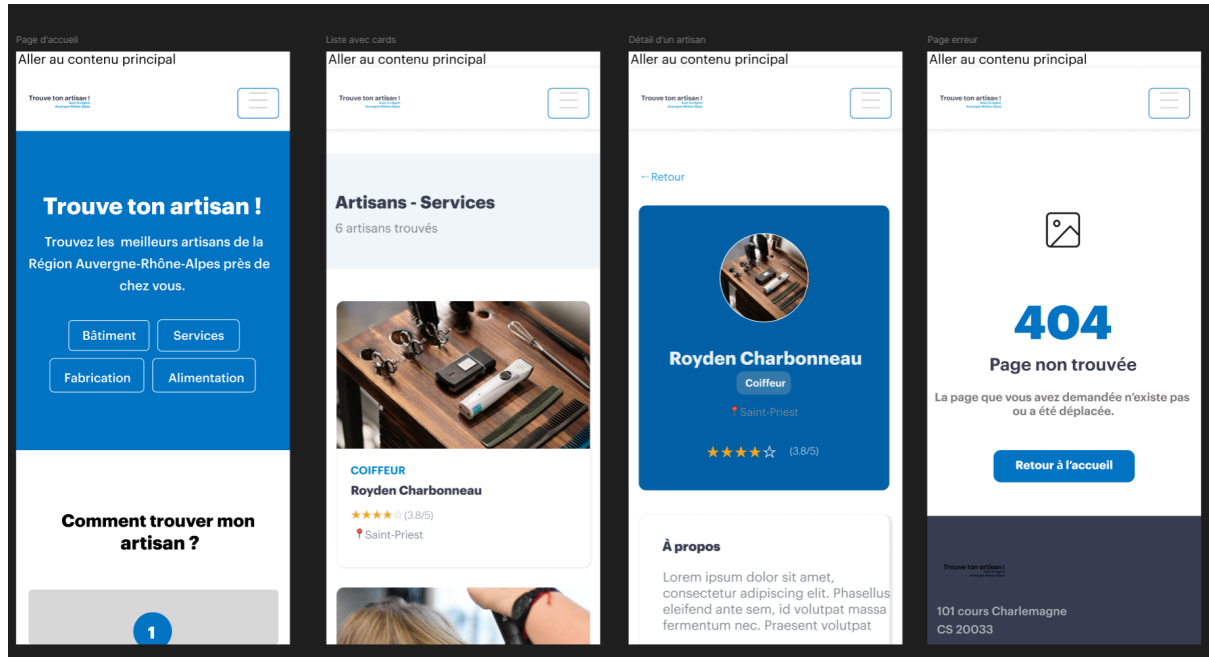
Livable	Description	Statut
Maquettes Figma	Mobile, tablette, desktop	✓ Réalisé
Code source frontend	React + Bootstrap + Sass	✓ Réalisé
API REST	Node.js + Express + Sequelize	✓ Réalisé
Scripts SQL	Création et alimentation de la BDD	✓ Réalisé
README.md	Prérequis et instructions d'installation	✓ Réalisé
Repository GitHub	Code source versionné	✓ Réalisé
Site en ligne	Hébergement sur Vercel	✓ Réalisé

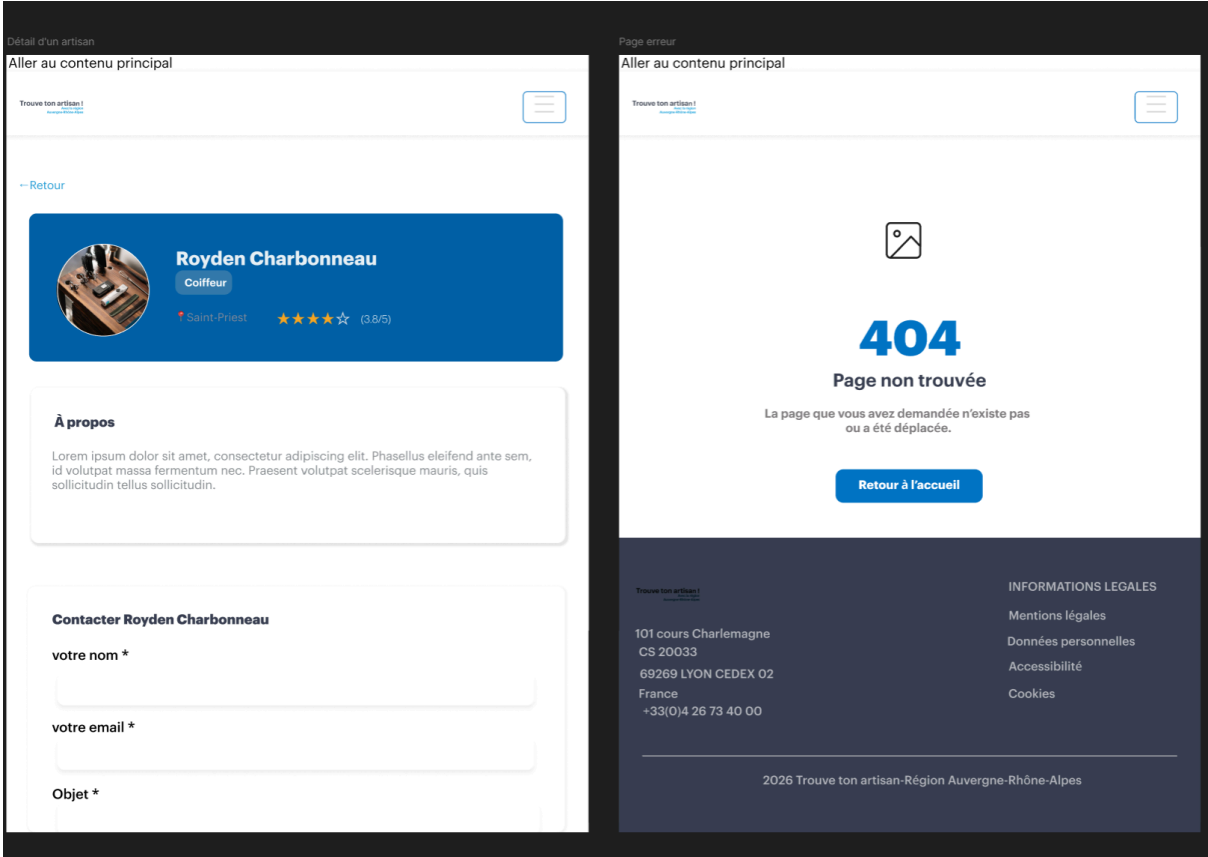
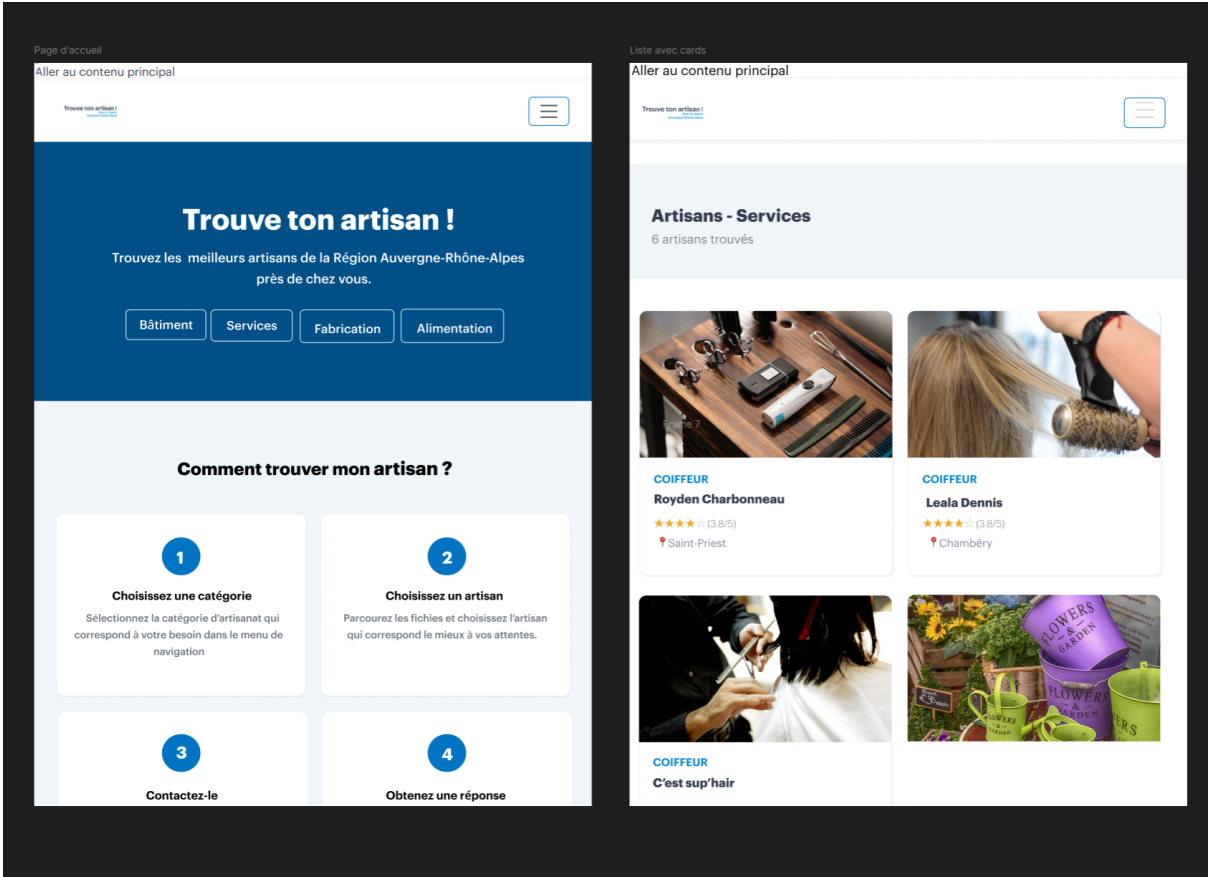
Dossier PDF

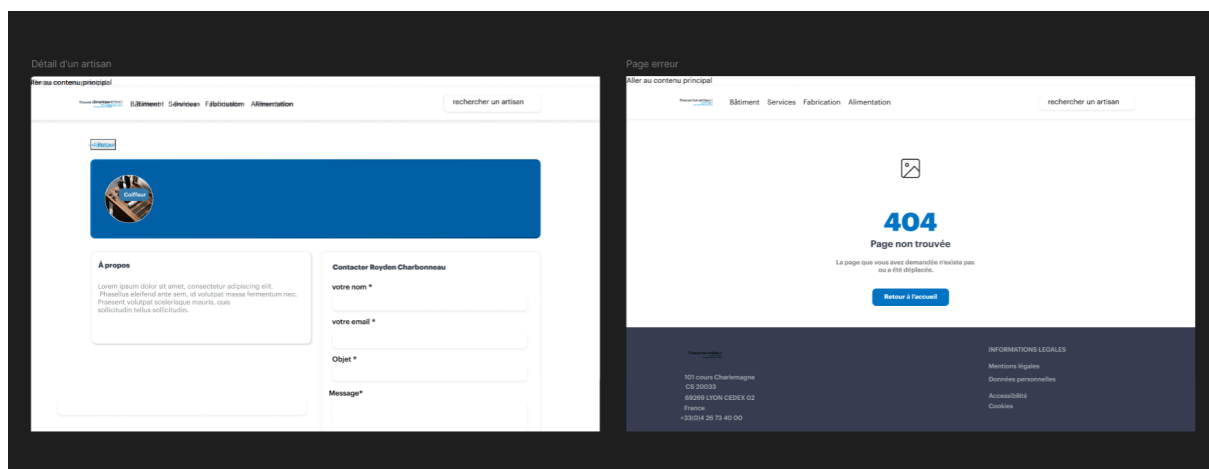
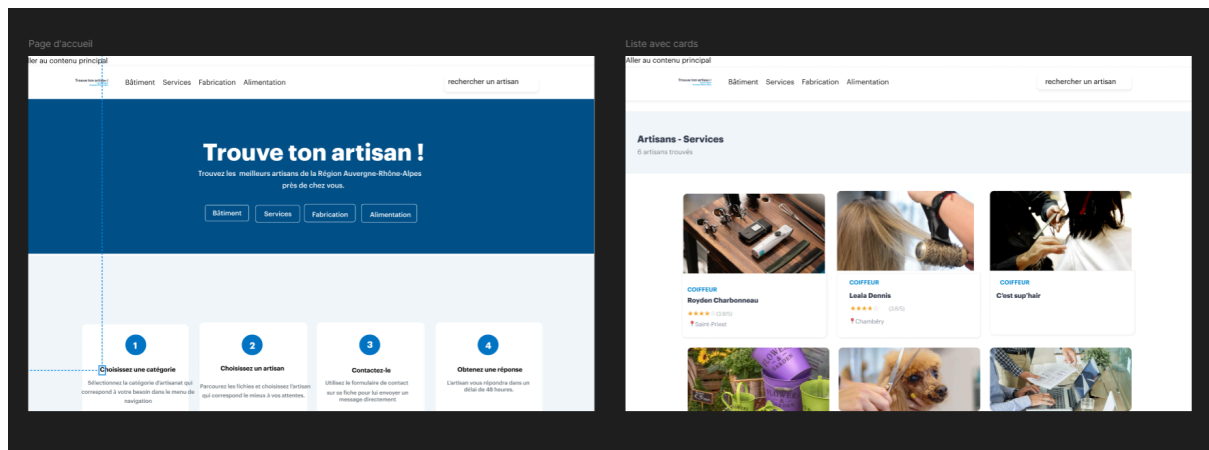
Ce document

✓ Réalisé

5. Maquettes Figma







- Page d'accueil
- Page liste des artisans
- Fiche artisan
- Page 404

Les maquettes ont été réalisées en respectant la charte graphique de la région Auvergne-Rhône-Alpes : police Graphik, palette de couleurs officielle (#0074c7 pour le bleu principal, #384050 pour le texte), et mise en page mobile first.

Lien vers les maquettes complètes Figma :

<https://www.figma.com/design/LNsBl2FIkMcFXysPRkcyEd/Figma-Trouve-ton-artisan?node-id=1-3&p=f&t=7TeDgxnNPSCALJuE-0>

6. Base de données

MCD — Modèle Conceptuel de Données

Le modèle conceptuel repose sur trois entités principales :

- **CATEGORIE** : regroupe les artisans par grand domaine (Bâtiment, Services, Fabrication, Alimentation)
- **SPECIALITE** : représente le métier précis d'un artisan (Plombier, Coiffeur, Boulanger...)
- **ARTISAN** : représente un professionnel avec toutes ses informations

Les règles de gestion sont les suivantes :

- Un artisan appartient à **une seule** spécialité
- Une spécialité est rattachée à **une seule** catégorie
- Une catégorie peut contenir **plusieurs** spécialités
- Une spécialité peut regrouper **plusieurs** artisans

text

CATEGORIE (1,n) ----- (1,1) SPECIALITE (1,n) ----- (1,1)
ARTISAN

MLD — Modèle Logique de Données

text

```
categories (  
  id          INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  nom         VARCHAR(100) NOT NULL UNIQUE,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)  
  
specialites (  
  id          INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  nom         VARCHAR(100) NOT NULL UNIQUE,  
  id_categorie INT UNSIGNED NOT NULL,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_categorie) REFERENCES categories(id)  
)  
  
artisans (  
  id          INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  nom         VARCHAR(150) NOT NULL,  
  id_specialite INT UNSIGNED NOT NULL,  
  note        DECIMAL(2,1) NOT NULL DEFAULT 0.0,
```

```

ville          VARCHAR(100) NOT NULL,
apropos        TEXT,
email          VARCHAR(255) NOT NULL,
site_web       VARCHAR(255),
image          VARCHAR(255),
top            BOOLEAN NOT NULL DEFAULT FALSE,
created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
FOREIGN KEY (id_specialite) REFERENCES specialites(id)
)

```

Vue SQL

Une vue `v_artisans_cards` a été créée pour faciliter les requêtes du frontend. Elle joint les trois tables et retourne pour chaque artisan son nom, sa spécialité, sa catégorie, sa note, sa ville, son image et son statut "top".

7. Sécurité — mesures mises en place

7.1 Helmet — En-têtes de sécurité HTTP

Mise en œuvre : Le module `helmet` est intégré dans le serveur Express via `app.use(helmet())`.

Intérêt : Helmet configure automatiquement une série d'en-têtes HTTP de sécurité qui protègent l'application contre les attaques courantes :

- `X-Content-Type-Options: nosniff` — empêche le navigateur d'interpréter les fichiers avec un type MIME différent de celui déclaré
- `X-Frame-Options: DENY` — empêche l'intégration du site dans une iframe (protection contre le clickjacking)
- `Strict-Transport-Security` — force l'utilisation de HTTPS
- `Content-Security-Policy` — restreint les sources de contenu autorisées

7.2 CORS — Contrôle des origines autorisées

Mise en œuvre : Le module `cors` est configuré pour n'autoriser que les origines connues :

```
javascript
```



```
app.use(cors({
  origin: ['http://localhost:5173'],
  methods: ['GET'],
  allowedHeaders: ['Content-Type', 'x-api-key']
}));
```

Intérêt : Sans configuration CORS, n'importe quel site web pourrait faire des requêtes vers l'API. En limitant les origines autorisées, on empêche les requêtes provenant de domaines non autorisés, ce qui protège contre les attaques de type CSRF (Cross-Site Request Forgery).

7.3 Rate Limiting — Limitation du nombre de requêtes

Mise en œuvre : Le module `express-rate-limit` est configuré pour limiter à 100 requêtes par fenêtre de 15 minutes par adresse IP :

javascript

```
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100
});
```

Intérêt : Cette mesure protège l'API contre les attaques par déni de service (DDoS) et les tentatives de force brute. Si un acteur malveillant tente d'envoyer des milliers de requêtes pour surcharger le serveur, il sera automatiquement bloqué après 100 requêtes.

7.4 Clé API — Restriction d'accès

Mise en œuvre : Un middleware vérifie la présence d'une clé API dans l'en-tête `x-api-key` de chaque requête :

javascript

```
const checkApiKey = (req, res, next) => {
  const apiKey = req.headers['x-api-key'];
  if (apiKey !== process.env.API_KEY) {
    return res.status(401).json({ message: 'Clé API invalide'
  });
  }
  next();
};
```

Intérêt : Le brief spécifie que l'accès à l'API doit être limité à l'application. La clé API permet de s'assurer que seul le frontend autorisé peut interroger l'API. La clé est stockée dans les variables d'environnement (fichier `.env`) et jamais exposée dans le code source.

7.5 Variables d'environnement — Protection des données sensibles

Mise en œuvre : Le fichier `.env` contient toutes les informations sensibles (identifiants BDD, clé API, port). Ce fichier est listé dans `.gitignore` et n'est donc jamais envoyé sur GitHub.

Intérêt : Cela évite d'exposer les credentials de la base de données et les clés secrètes dans le code source public. Un attaquant qui accèderait au repository GitHub ne pourrait pas obtenir ces informations.

7.6 Validation des données — Sequelize

Mise en œuvre : Le modèle Sequelize `Artisan` intègre des validateurs natifs :

- `isEmail: true` sur le champ email
- `isUrl: true` sur le champ `site_web`
- `min: 0, max: 5` sur le champ note

Intérêt : La validation côté serveur garantit que les données insérées en base de données respectent le format attendu, indépendamment du frontend. Cela protège contre les injections de données malformées.

7.7 Paramètres préparés — Protection contre les injections SQL

Mise en œuvre : Toutes les requêtes à la base de données passent par Sequelize ORM, qui utilise des requêtes paramétrées.

Intérêt : Les injections SQL sont l'une des vulnérabilités les plus répandues (classée #1 dans l'OWASP Top 10). En utilisant un ORM avec des paramètres liés, les entrées utilisateur ne sont jamais interprétées comme du code SQL, ce qui rend les injections impossibles.

8. Veille sur les vulnérabilités de sécurité

Méthodologie de veille

Durant le développement du projet, une veille active sur les vulnérabilités de sécurité web a été effectuée en consultant :

- L'**OWASP Top 10** (Open Web Application Security Project), référence mondiale des 10 risques de sécurité les plus critiques pour les applications web
- Les **bulletins de sécurité npm** via la commande `npm audit`
- La base de données **CVE** (Common Vulnerabilities and Exposures)

Vulnérabilités identifiées et mesures correctives

Injection SQL (OWASP A03:2021)

Cette vulnérabilité permet à un attaquant d'insérer du code SQL malveillant dans les champs de saisie pour manipuler la base de données. Elle a été corrigée par l'utilisation de Sequelize ORM avec des requêtes paramétrées, qui sépare strictement les données du code SQL.

Cross-Site Scripting — XSS (OWASP A03:2021)

Les attaques XSS consistent à injecter du code JavaScript malveillant dans les pages web. React protège nativement contre le XSS en échappant automatiquement toutes les valeurs affichées dans le JSX. De plus, Helmet configure l'en-tête `Content-Security-Policy` pour restreindre les sources de scripts autorisées.

Exposition de données sensibles (OWASP A02:2021)

Les identifiants de base de données et clés secrètes sont stockés dans des variables d'environnement (fichier `.env`) exclu du dépôt Git via `.gitignore`. Ainsi, aucune donnée sensible n'est exposée dans le code source public.

Broken Access Control (OWASP A01:2021)

L'accès à l'API est contrôlé par une clé API transmise dans les en-têtes HTTP. En production, toute requête sans clé valide est rejetée avec un code HTTP 401. Cela empêche l'accès non autorisé aux données.

Déni de service (DDoS)

Un système de rate limiting (100 requêtes / 15 minutes / IP) protège l'API contre les attaques par surcharge. Les tentatives de spam ou d'automatisation malveillante sont automatiquement bloquées.

Résultat npm audit

L'audit des dépendances npm a révélé 2 vulnérabilités de sévérité modérée dans des dépendances de développement. Ces vulnérabilités n'affectent pas l'environnement de production et ne présentent pas de risque pour les utilisateurs finaux.

9. Liens du projet

Ressource	Lien
Site en ligne	https://trouve-ton-artisan-plum.vercel.app
Repository GitHub	https://github.com/meliokinchi/Trouve-ton-artisan
Script de création BDD	github.com/meliokinchi/Trouve-ton-artisan/blob/main/api/scripts/schema_exemple.sql
Script d'alimentation BDD	github.com/meliokinchi/Trouve-ton-artisan/blob/main/api/scripts/seed_exemple.sql